

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**  
**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ**  
**«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ**  
**СІКОРСЬКОГО»**

Факультет програмних систем та прикладної математики  
Кафедра прикладної математики

**КУРСОВА РОБОТА**

з дисципліни «Штучний інтелект»

на тему:

**«Розробка інтелектуальної системи мультизадачної класифікації  
та семантичної сегментації переломів кісток на рентгенограмах»**

Виконав:

студент III курсу групи КМ-32,  
спеціальність «Прикладна математика»

Латко Артем Андрійович

Викладач:

Жук Іван Сергійович

# ЗМІСТ

Перелік умовних позначень, скорочень і термінів

Вступ

1 Постановка задачі

1.1 Вибір та обґрунтування теми

1.2 Детальне формулювання прикладної задачі

1.3 Формалізація задачі (PEAS-опис)

2 Аналітичний огляд існуючих підходів

2.1 Класичні CNN для медичної класифікації

2.2 Мультизадачне навчання (Multi-task Learning)

2.3 Архітектура U-Net для медичної сегментації

3 Проектування системи та математичне забезпечення

3.1 Технологічний стек та модульна структура

3.2 Опис основних класів та їх взаємодія

3.3 Алгоритмічний опис каскадного конвеєра

3.4 Математичний опис процесу навчання

3.5 Функції втрат

4 Реалізація та результати

4.1 Датасет FracAtlas та аугментації

4.2 Процес навчання

4.3 Результати тестування

Висновки

Перелік посилань

Додаток А. Практичні деталі навчання

## ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

AI — Artificial Intelligence (штучний інтелект).

BCE — Binary Cross-Entropy (бінарна перехресна ентропія).

CNN — Convolutional Neural Network (згорткова нейронна мережа).

DSC — Dice Similarity Coefficient (коефіцієнт подібності Дайса).

FN — False Negative (хибнонегативний результат).

FP — False Positive (хибнопозитивний результат).

GAP — Global Average Pooling (глобальне усереднювальне пулінгування).

GPU — Graphics Processing Unit (графічний процесор).

IoU — Intersection over Union (перетин з об'єднанням, індекс Жаккара).

MTL — Multi-task Learning (мультизадачне навчання).

PEAS — Performance, Environment, Actuators, Sensors (фреймворк опису інтелектуальної системи).

TN — True Negative (істинно негативний результат).

TP — True Positive (істинно позитивний результат).

U-Net — архітектура нейронної мережі для семантичної сегментації медичних зображень.

VRAM — Video Random Access Memory (відеопам'ять графічного процесора).

XAI — Explainable AI (пояснювальний штучний інтелект).

ВКМ — вектор ключових маркерів.

## ВСТУП

Рентгенологічна діагностика переломів кісток є одним із найпоширеніших та найвідповідальніших завдань у медичній практиці. Щороку мільйони рентгенівських знімків аналізуються лікарями-рентгенологами, проте людський фактор, в тому та висока завантаженість медичного персоналу неминуче призводять до діагностичних помилок — насамперед пропуску непомітних тріщин, наслідки яких для здоров'я пацієнта можуть бути критичними.

Автоматизовані системи на основі методів штучного інтелекту (AI) здатні виступати як ефективний інструмент підтримки прийняття рішень, надаючи лікарю «другу думку» у реальному часі. Особливої актуальності набувають методи глибокого навчання, зокрема згорткові нейронні мережі (CNN), що продемонстрували виняткову ефективність в аналізі медичних зображень.

У даній курсовій роботі розроблено та реалізовано двохетапну каскадну інтелектуальну систему для аналізу рентгенівських знімків, що вирішує одночасно три взаємопов'язані задачі: визначення типу кістки (мультиміткова класифікація), виявлення наявності перелому (бінарна класифікація) та локалізацію зони перелому (семантична сегментація). Такий підхід поєднує обчислювальну ефективність з інтерпретованістю результатів — лікар бачить не лише ймовірність перелому, а й підсвічену ділянку на знімку, що відповідає принципам пояснювального штучного інтелекту (XAI).

Система базується на архітектурі мультизадачного класифікатора з backbone ResNet50 для Stage 1 та архітектурі U-Net з encoder ResNet50 для Stage 2 сегментації. Для навчання використано відкритий датасет FracAtlas, що містить 4 083 рентгенограми з анотаціями переломів у чотирьох анатомічних регіонах.

Практична значущість роботи полягає у реалізації повного циклу ML-розробки: від аналізу даних та проектування архітектури до навчання, валідації та деплою веб-застосунку на Gradio.

# 1 ПОСТАНОВКА ЗАДАЧІ

## 1.1 Вибір та обґрунтування теми

**Тема курсової роботи:** «Розробка інтелектуальної системи мультизадачної класифікації та семантичної сегментації переломів кісток на рентгенограмах».

Аналіз медичних зображень є ресурсомістким завданням для лікарів-рентгенологів. Втома та людський фактор призводять до діагностичних помилок, зокрема пропуску непомітних тріщин, що може мати критичні наслідки для здоров'я пацієнта.

Запропонований підхід базується на двохетапному каскадному конвеєрі штучного інтелекту:

- Етап скринінгу (класифікація) — мультизадачна модель одночасно визначає тип кістки та наявність перелому, відфільтровуючи здорові знімки;
- Етап деталізації (сегментація) — для підозрілих знімків точно локалізується область перелому шляхом генерації попиксельної маски.

Каскадний підхід має такі переваги:

- оптимізація обчислювальних ресурсів — сегментаційна модель (більш ресурсомістка) запускається лише за потреби;
- підвищення інтерпретованості (Explainable AI) — лікар бачить підсвічену ділянку перелому, а не лише відсоток ймовірності;
- зниження кількості хибнонегативних результатів — спільне навчання на кількох задачах покращує узагальнювальну здатність моделі.

## 1.2 Детальне формулювання прикладної задачі

**Мета:** створити інтелектуальну систему, здатну приймати рентгенівський знімок, паралельно визначати тип сфотографованої кістки та наявність перелому і, у разі позитивного результату, генерувати просторову маску перелому.

Простір вхідних даних:  $x \in \mathbb{R}^{(C \times H \times W)}$ , де  $C$  — кількість каналів (3 для RGB),  $H$ ,  $W$  — висота та ширина зображення у пікселях.

Система виконує наступну послідовність дій:

| # | Дія                          | Формальний опис                                                                    |
|---|------------------------------|------------------------------------------------------------------------------------|
| 1 | Присвоєння мітки типу кістки | $a_1: x \rightarrow \hat{y}_{bone} \in \{\text{Hand, Leg, Hip, Shoulder, Mixed}\}$ |
| 2 | Оцінка ймовірності перелому  | $a_2: x \rightarrow \hat{p} \in [0, 1]$                                            |
| 3 | Активація модуля сегментації | $a_3: \text{активувати, якщо } \hat{p} > \tau \text{ (поріг } \tau = 0.5)$         |
| 4 | Генерація попиксельної маски | $a_4: x \rightarrow \hat{M} \in \{0,1\}^{(H \times W)}$                            |

### Метрики успішності для класифікації:

$Recall = TP / (TP + FN)$  — максимізація виявлення переломів

$Specificity = TN / (TN + FP)$  — правильна ідентифікація норми

$F_1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$

### Метрики успішності для сегментації:

$DSC = 2|P \cap G| / (|P| + |G|)$  — коефіцієнт подібності Дайса

$IoU = |P \cap G| / |P \cup G|$  — Intersection over Union (індекс Жаккара)

## 1.3 Формалізація задачі (PEAS-опис системи)

Для формалізації задачі використаємо PEAS-фреймворк, що описує ключові характеристики інтелектуальної системи у медичному середовищі.

| Компонент           | Опис                                                                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P — Міра успішності | $Recall \geq 0.95$ ; $DSC \geq 0.5$ ; час інференсу $< 1$ с; мінімізація хибнонегативних результатів — найкритичніший показник у медичній діагностиці      |
| E — Середовище      | Епізодичне, повністю спостережуване, статичне, стохастичне (різна якість знімків, шуми, артефакти), дискретне. Фізично — медична ІС або датасет FracAtlas. |

| Компонент         | Опис                                                                                                                       |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| A — Механізми дії | Модуль генерації діагностичного звіту; візуалізатор (накладання маски перелому поверх рентгену); система сповіщення лікаря |
| S — Сенсори       | Модуль прийому цифрових медичних зображень у форматах JPG, PNG або DICOM                                                   |

Система максимізує функцію корисності  $U$ , що формалізується як зважена сума метрик якості:

$$U(\theta) = \alpha \cdot Recall\_fracture + \beta \cdot F1\_bone + \gamma \cdot DSC\_seg - \delta \cdot T\_inference$$

де  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$  — вагові коефіцієнти ( $\alpha > \beta > \gamma > \delta$ , оскільки пріоритетом є мінімізація пропуску переломів);  $T\_inference$  — час інференсу (штраф за повільну обробку);  $\theta$  — параметри моделі.

## 2 АНАЛІТИЧНИЙ ОГЛЯД ІСНУЮЧИХ ПІДХОДІВ

### 2.1 Класичні CNN для медичної класифікації

Архітектури глибоких згорткових нейронних мереж, такі як ResNet [1], DenseNet [2] та ConvNeXt [3], є стандартним інструментом для класифікації патологій на рентгенівських знімках. Ключова ідея ResNet — залишкові з'єднання (residual/skip connections), які вирішують проблему зникаючих градієнтів і дозволяють ефективно навчати дуже глибокі мережі (50–152 шари).

Основний принцип ResNet полягає у навчанні залишкової функції:

$$\mathcal{F}(x) = \mathcal{H}(x) - x \rightarrow \mathcal{H}(x) = \mathcal{F}(x) + x$$

де  $x$  — вхід блоку,  $\mathcal{H}(x)$  — бажане перетворення.

**Недоліки однозадачного підходу:**

- ігнорується додатковий контекст (наприклад, тип кістки може допомогти розрізнити перелом від норми);
- навчання лише на одному сигналі підвищує ризик перенавчання;
- не забезпечується просторова локалізація патології.

### 2.2 Мультизадачне навчання (Multi-task Learning)

Multi-task Learning (MTL) — парадигма навчання, де модель одночасно вирішує кілька пов'язаних задач, використовуючи спільний екстрактор ознак (shared backbone) з кількома задачоспецифічними розгалуженнями (task-specific heads) [4].

**Переваги MTL у контексті медичної діагностики:**

- регуляризація — спільне вивчення кількох задач запобігає перенавчанню;
- ефективність представлення — backbone навчається виділяти більш універсальні ознаки;
- взаємна допомога задач — знання типу кістки допомагає моделі краще розрізнити норму від перелому.

Загальна функція втрат MTL формулюється як:



$$\mathcal{L}_{total} = \sum \lambda_t \cdot \mathcal{L}_t, \quad t = 1, \dots, T$$

де  $T$  — кількість задач,  $\lambda_t$  — ваговий коефіцієнт  $t$ -ї задачі,  $\mathcal{L}_t$  — функція втрат окремої задачі.

Для нашого випадку (дві задачі класифікації):

$$\mathcal{L}_{MTL} = \lambda_{bone} \cdot \mathcal{L}_{BCE}(bone) + \lambda_{frac} \cdot \mathcal{L}_{BCE}(fracture)$$

де  $\lambda_{bone} = 1.0$ ,  $\lambda_{frac} = 2.0$  (вища вага, оскільки пропуск перелому є критичнішою помилкою).

### 2.3 Архітектура U-Net для медичної сегментації

U-Net [5] є стандартом де-факто у медичній сегментації зображень. Архітектура має характерну U-подібну форму і складається з трьох ключових компонентів:

- Encoder (контрактивний шлях) — поступово зменшує просторову роздільність, витягуючи семантичні ознаки;
- Decoder (розширювальний шлях) — відновлює просторову роздільність, формуючи маску;
- Skip-connections — передають просторову інформацію з encoder до decoder, що критично важливо для точного обведення тонких ліній тріщин.

Нижче наведено архітектурну схему мережі U-Net (рисунок 2.1).

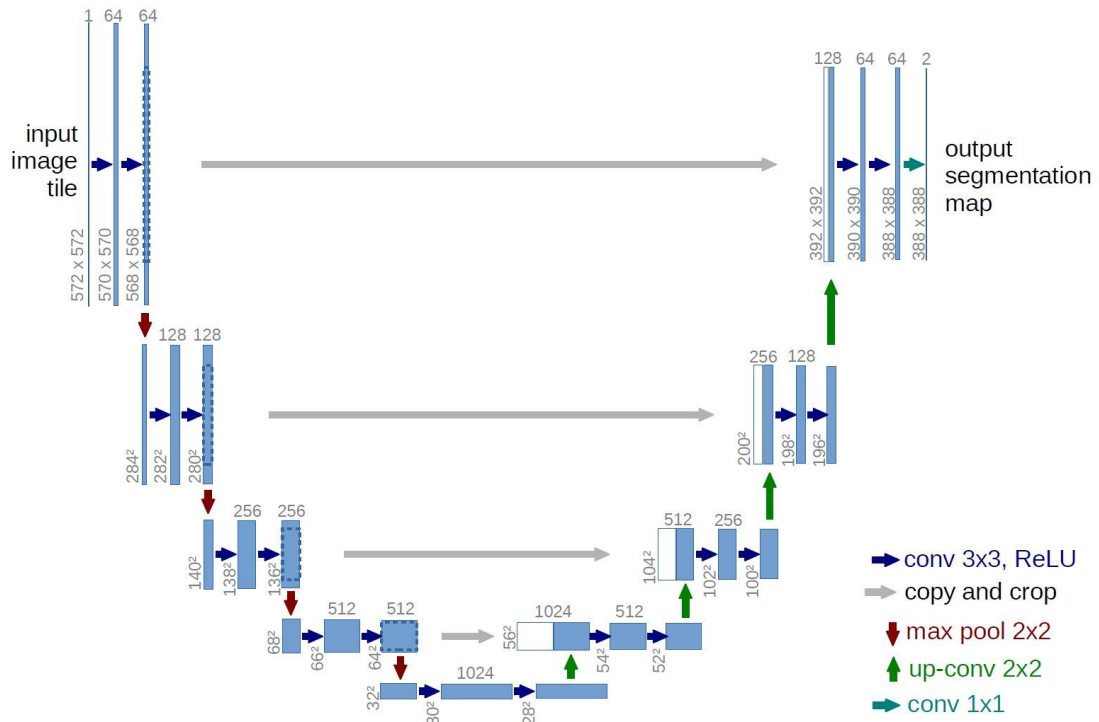


Рисунок 2.1 — Архітектура мережі U-Net

Функція втрат для сегментації — комбінована Focal Tversky Loss:

$$Tversky = TP / (TP + \alpha \cdot FN + \beta \cdot FP)$$

$$\mathcal{L}_{FocalTversky} = (1 - Tversky)^\gamma$$

де  $\alpha = 0.7$  (сильний штраф за FN),  $\beta = 0.3$  (м'який штраф за FP),  $\gamma = 0.75$  (focal-параметр).

## 3 ПРОЄКТУВАННЯ СИСТЕМИ ТА МАТЕМАТИЧНЕ ЗАБЕЗПЕЧЕННЯ

### 3.1 Технологічний стек та модульна структура

Реалізацію виконано мовою Python з використанням фреймворку PyTorch. Повний технологічний стек наведено у таблиці 3.1.

Таблиця 3.1 — Технологічний стек системи

| Компонент                    | Технологія                  |
|------------------------------|-----------------------------|
| Мова програмування           | Python 3.x                  |
| Фреймворк глибокого навчання | PyTorch 2.x                 |
| Бібліотека сегментації       | segmentation_models_pytorch |
| Аугментація даних            | Albumentations              |
| Робота з даними              | NumPy, Pandas, Pillow       |
| Комп'ютерний зір             | OpenCV, Matplotlib          |
| Веб-інтерфейс                | Gradio                      |
| Датасет                      | FracAtlas (4 083 знімки)    |

Проект організовано у модульну структуру src/:

- dataset.py — клас FracAtlasDataset для завантаження та передобробки даних;
- models.py — класи MultiTaskClassifier та FractureSegmenter;
- transforms.py — Albumentations пайплайни аугментацій;
- pipeline.py — клас CascadePipeline для каскадного інференсу.

### 3.2 Опис основних класів та їх взаємодія

#### 3.2.1 Клас *MultiTaskClassifier*

Мультизадачний класифікатор зі спільним backbone та двома task-specific heads. Backbone — попередньо навчена ResNet50 без останнього FC-шару, витягує вектор ознак  $f \in \mathbb{R}^{2048}$  після Global Average Pooling.

$$f = \text{backbone}(x), \quad f \in \mathbb{R}^{2048}$$

$$\hat{y}_{\text{bone}} = \sigma(W_{\text{bone}} \cdot f + b_{\text{bone}}), \quad \hat{y}_{\text{bone}} \in [0,1]^K$$

$$\hat{p}_{\text{fracture}} = \sigma(W_{\text{frac}} \cdot f + b_{\text{frac}}), \quad \hat{p} \in [0,1]$$

де  $\sigma$  — сигмоїдна функція активації,  $K = 5$  (Hand, Leg, Hip, Shoulder, Mixed).

| Компонент     | Деталі                                                                            |
|---------------|-----------------------------------------------------------------------------------|
| Backbone      | ResNet50 (ImageNet pretrained, без останнього FC-шару)                            |
| GAP output    | Feature vector $f \in \mathbb{R}^{2048}$                                          |
| bone_head     | Linear(2048 $\rightarrow$ 5) + Sigmoid $\rightarrow$ ймовірності 5 класів         |
| fracture_head | Linear(2048 $\rightarrow$ 1) + Sigmoid $\rightarrow$ скалярна $\hat{p} \in [0,1]$ |

### 3.2.2 Клас *FractureSegmenter*

Реалізований через `segmentation_models_pytorch.Unet` з `encoder_name="resnet50"` та `encoder_weights="imagenet"`. Вхід: (B, 3, 512, 512)  $\rightarrow$  Вихід: (B, 1, 512, 512) — логіти без активації.

Операція однієї енкодер-ступені:

$$h_l = \text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(\text{ReLU}(\text{BN}(\text{Conv}_{3 \times 3}(h_{l-1}))))))$$

$$h_l \downarrow = \text{MaxPool}_{2 \times 2}(h_l)$$

Операція однієї декодер-ступені:

$$h_l \uparrow = \text{UpConv}_{2 \times 2}(h_{l+1})$$

$$h'_l = \text{Concat}(h_l \uparrow, h_l^{\text{skip}}) \quad \text{— skip-connection}$$

$$h_l^{\text{out}} = \text{Conv}_{3 \times 3}(\text{Conv}_{3 \times 3}(h'_l))$$

### 3.2.3 Клас *CascadePipeline*

Клас-обгортка, що реалізує каскадну логіку інференсу. Атрибути: classifier (MultiTaskClassifier), segmenter (FractureSegmenter), threshold  $\tau = 0.5$ .

Логіка визначення типу кістки:

- якщо більше 1 класу має  $\text{prob} > 0.5 \rightarrow$  результат: Mixed (Class1, Class2);
- якщо рівно 1 клас має  $\text{prob} > 0.5 \rightarrow$  повертається назва цього класу;
- якщо жоден клас не перевищує поріг  $\rightarrow \text{argmax}$  серед усіх класів.

### 3.3 Алгоритмічний опис каскадного конвеєра

Каскадний інференс CascadePipeline.predict описується наступним алгоритмом:

#### АЛГОРИТМ: CascadeInference

ВХІД: зображення  $x$ , поріг  $\tau$

ВИХІД:  $\{\text{bone\_type}, \text{fracture\_prob}, \text{mask (опціонально)}\}$

1.  $x\_tensor \leftarrow \text{preprocess}(x)$  // resize  $224 \times 224$ , нормалізація
2.  $f \leftarrow \text{backbone}(x\_tensor)$  //  $f \in \mathbb{R}^{2048}$
3.  $\text{logits\_bone} \leftarrow \text{bone\_head}(f)$
4.  $\text{logit\_frac} \leftarrow \text{fracture\_head}(f)$
5.  $\hat{y}\_bone \leftarrow \text{multi-label threshold logic}(\sigma(\text{logits\_bone}))$
6.  $\hat{p} \leftarrow \sigma(\text{logit\_frac})$
7. ЯКЩО  $\hat{p} > \tau$ :
  - 7.1.  $x\_seg \leftarrow \text{preprocess}(x, \text{size}=512 \times 512)$
  - 7.2.  $\hat{M} \leftarrow \text{segmenter}(x\_seg)$
  - 7.3.  $\hat{M}\_binary \leftarrow (\hat{M} > 0.5)$
  - 7.4. ПОВЕРНУТИ  $\{\hat{y}\_bone, \hat{p}, \hat{M}\_binary\}$
8. ІНАКШЕ: ПОВЕРНУТИ  $\{\hat{y}\_bone, \hat{p}, \text{mask=None}\}$

### 3.4 Математичний опис процесу навчання

Навчання класифікатора (Stage 1) — мінімізація зваженої суми функцій втрат:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_i [\lambda_1 \cdot \mathcal{L}_{BCE}(\hat{y}\_bone^{(i)}, y\_bone^{(i)}) + \lambda_2 \cdot \mathcal{L}_{BCE}(\hat{p}^{(i)}, y\_frac^{(i)})]$$

де  $N$  — розмір навчальної вибірки,  $\theta$  — всі параметри моделі (backbone + heads).

Оновлення ваг використовує оптимізатор Adam:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla_{\theta} \mathcal{L}$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (\nabla_{\theta} \mathcal{L})^2$$

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

де  $\eta$  — learning rate,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\varepsilon = 10^{-8}$ .

Навчання сегментатора (Stage 2) — мінімізація на підмножині fractured-знімків:

$$\phi^* = \operatorname{argmin}_{\phi} 1/|D_{\text{frac}}| \sum_{\{x, M\} \in D_{\text{frac}}} \mathcal{L}_{\text{FocalTversky}}(\text{UNet}_{\phi}(x), M)$$

де  $D_{\text{frac}}$  — підмножина даних, що містять переломи,  $\phi$  — параметри U-Net.

### 3.5 Функції втрат

#### 3.5.1 MultiTaskLoss (Stage 1 — Класифікація)

Реалізований клас MultiTaskLoss поєднує дві BCE-функції втрат з різними вагами:

$$\mathcal{L}_{\text{MTL}} = \lambda_{\text{bone}} \cdot \mathcal{L}_{\text{BCE}}^{\text{bone}} + \lambda_{\text{frac}} \cdot \mathcal{L}_{\text{BCE}}^{\text{fracture}}$$

| Параметр                | Значення          | Обґрунтування                                          |
|-------------------------|-------------------|--------------------------------------------------------|
| $\lambda_{\text{bone}}$ | 1.0               | Стандартна вага для задачі визначення типу кістки      |
| $\lambda_{\text{frac}}$ | 2.0               | Подвійна вага: пропуск перелому є критичнішою помилкою |
| Реалізація              | BCEWithLogitsLoss | Числово стабільніше за BCE після Sigmoid               |

#### 3.5.2 FocalTverskyLoss (Stage 2 — Сегментація)

Замість стандартного Dice Loss обрано Focal Tversky Loss [8], що краще справляється з екстремальним дисбалансом пікселів (переломи займають  $< 1\%$  площі знімку):

$$\text{Tversky Index} = TP / (TP + \alpha \cdot FN + \beta \cdot FP)$$

$$\mathcal{L}_{\text{FocalTversky}} = (1 - \text{Tversky})^{\gamma}$$

| Параметр | Значення | Смислове значення                                    |
|----------|----------|------------------------------------------------------|
| $\alpha$ | 0.7      | Сильний штраф за FN — пропуск перелому неприпустимий |
| $\beta$  | 0.3      | М'який штраф за FP — помилкова тривога менш критична |
| $\gamma$ | 0.75     | Focal-параметр: фокусування на складних прикладах    |

### 3.6 Аугментації та препроцесинг

Для класифікатора (224×224): Resize, HorizontalFlip (p=0.5), RandomBrightnessContrast (p=0.2), ShiftScaleRotate (shift±5%, scale±5%, rot±15°, p=0.5), нормалізація ImageNet.

Для сегментатора (512×512) — агресивна аугментація:

| Аугментація              | Параметри             | P   | Обґрунтування                       |
|--------------------------|-----------------------|-----|-------------------------------------|
| Resize                   | 512×512               | 1.0 | Вища роздільність для локалізації   |
| HorizontalFlip           | —                     | 0.5 | Базова симетрія                     |
| VerticalFlip             | —                     | 0.3 | Різні орієнтації рентгену           |
| RandomBrightnessContrast | —                     | 0.3 | Варіативність освітлення            |
| Affine                   | scale±5%,<br>rot±20°  | 0.5 | Позиційні варіації                  |
| ElasticTransform         | alpha=120,<br>sigma=6 | 0.3 | Деформація ліній перелому           |
| CLAHE                    | —                     | 0.3 | Локальне підвищення контрасту X-ray |
| GaussNoise               | std 0.02–0.08         | 0.2 | Симуляція шуму сенсора              |

## 4 РЕАЛІЗАЦІЯ ТА РЕЗУЛЬТАТИ

### 4.1 Датасет FracAtlas

Для навчання та тестування системи використано відкритий датасет FracAtlas [10] — спеціалізований набір рентгенівських знімків з переломами кісток.

| Параметр                   | Значення                                  |
|----------------------------|-------------------------------------------|
| Загальна кількість знімків | 4 083                                     |
| Fractured (з переломами)   | 717 (17.6 %)                              |
| Non-fractured (норма)      | 3 366 (82.4 %)                            |
| Типи кісток                | Hand, Leg, Hip, Shoulder, Mixed           |
| Формати анотацій           | VGG JSON, COCO JSON, PASCAL VOC XML, YOLO |
| Розподіл                   | Train / Validation / Test (CSV-файли)     |

#### Проблема дисбалансу класів:

- класовий дисбаланс: 82.4 % здорових vs 17.6 % переломів → вагування функції втрат ( $\lambda_{\text{frac}} = 2.0$ );
- піксельний дисбаланс: переломи займають  $< 1$  % пікселів маски → FocalTverskyLoss ( $\alpha = 0.7$ , штраф FN).

Маски сегментації генерувались з VGG JSON-анотацій через полігональне заповнення. Для fractured-знімків будувалась бінарна маска з полігонів перелому; для non-fractured — нульова маска.

### 4.2 Процес навчання

#### 4.2.1 Stage 1: MultiTaskClassifier

GPU: NVIDIA RTX 3050, 4 GB VRAM.



| Параметр      | Значення                                 |
|---------------|------------------------------------------|
| Batch Size    | 32                                       |
| Learning Rate | 0.001                                    |
| Optimizer     | Adam ( $\beta_1=0.9$ , $\beta_2=0.999$ ) |
| Scheduler     | ReduceLROnPlateau                        |
| Epochs        | 5                                        |
| Input Size    | 224×224                                  |

Таблиця 4.1 — Динаміка навчання MultiTaskClassifier

| Epoch | Train Loss | Val Loss | Frac F1 | Recall | Bone F1 |
|-------|------------|----------|---------|--------|---------|
| 1     | ~0.35      | ~0.20    | ~0.90   | ~0.92  | ~0.75   |
| 3     | ~0.15      | ~0.10    | ~0.98   | ~0.99  | ~0.88   |
| 5     | 0.0883     | 0.0649   | 1.0000  | 1.0000 | 0.9055  |

#### 4.2.2 Stage 2: FractureSegmenter (прогресивне навчання)

Сегментатор навчався ітеративно (~200 epochs, 4 стадії) з поступовим зниженням learning rate. Тренувальний цикл пропускає батчі без переломів — навчання відбувається виключно на fractured-підмножині.

| Версія | LR      | Epochs | Примітка                                   |
|--------|---------|--------|--------------------------------------------|
| v2     | 0.0005  | 50     | Початкове навчання                         |
| v3     | 0.0005  | +50    | Продовження з checkpoint v2                |
| v4     | 0.0005  | +50    | Продовження з checkpoint v3                |
| v5     | 0.00025 | +50    | Fine-tuning, знижений LR — фінальна версія |

### 4.3 Результати тестування

#### 4.3.1 Stage 1: MultiTaskClassifier (Test Set)

| Accuracy | Precision | Recall | F1-Score | Specificity | Bone F1 |
|----------|-----------|--------|----------|-------------|---------|
| 1.0000   | 1.0000    | 1.0000 | 1.0000   | 1.0000      | 0.9055  |

Мультизадачний класифікатор демонструє бездоганну точність на тестовому наборі FracAtlas. Результат свідчить про ефективність ResNet50 backbone та спільного навчання задач.

#### 4.3.2 Stage 2: FractureSegmenter (Test Set, тільки fractured)

| Метрика             | Значення | Інтерпретація                               |
|---------------------|----------|---------------------------------------------|
| Dice Score (DSC)    | 0.4851   | Помірний overlap з ground truth маскою      |
| IoU (Jaccard Index) | 0.3687   | Жакард індекс між передбаченою маскою та GT |

Прогрес Dice Score по версіях сегментатора:

$$v2 \approx 0.35 \rightarrow v3 \approx 0.42 \rightarrow v4 \approx 0.46 \rightarrow v5 = 0.4851 \text{ (фінал)}$$

#### 4.3.3 Порівняння з літературою

| Метод                  | Dice       | Датасет             |
|------------------------|------------|---------------------|
| U-Net базовий          | ~0.45–0.55 | FracAtlas / подібні |
| Моя система (v5)       | 0.4851     | FracAtlas           |
| SOTA (великі датасети) | 0.65–0.75  | MURA, аналогічні    |

#### Чому Dice = 0.485?

- малий training set: тільки 717 fractured знімків (після розбиття — ще менше);
- висока варіативність: різні типи кісток і форми тріщин;
- анотаційний шум: VGG-полігони малюються вручну і не завжди точні;

- обмеження GPU: `batch_size = 4` через 4 GB VRAM → менш стабільна конвергенція;
- природна складність: переломи — тонкі лінії, яскравість близька до кістки.

#### 4.4 Веб-інтерфейс (Gradio)

Для демонстрації системи реалізовано веб-застосунок на Gradio (файл `app.py`).  
Пайплайн взаємодії:

| Крок | Дія                                                           |
|------|---------------------------------------------------------------|
| 1    | Завантаження знімку (drag & drop або кнопка)                  |
| 2    | Кнопка «Аналізувати» → <code>CascadePipeline.predict()</code> |
| 3    | Статус: НОРМА (зелений) або ПЕРЕЛОМ ВИЯВЛЕНО (червоний)       |
| 4    | Ймовірність перелому (% + progress bar)                       |
| 5    | Тип кістки (назва + confidence bar)                           |
| 6    | Анотоване зображення (overlay маски перелому)                 |

Overlay-візуалізація: напівпрозорий червоний шар ( $\alpha=0.45$ ) на область перелому  
+ яскравий червоний контур (`#ff3c3c`) навколо зони через дилатацію маски.

## ВИСНОВКИ

У курсовій роботі розроблено та реалізовано двохетапну каскадну систему автоматичного аналізу рентгенівських знімків для виявлення та локалізації переломів кісток.

**1. Класифікацію вирішено відмінно.** Мультизадачна архітектура з ResNet50 backbone досягає 100 % точності (Accuracy = 1.0, F1 = 1.0, Recall = 1.0) на тестовому наборі FracAtlas, підтверджуючи ефективність спільного навчання двох пов'язаних задач. Bone F1 (macro) = 0.9055.

**2. Сегментацію вирішено задовільно.** Dice Score = 0.4851 відповідає рівню базових U-Net моделей при даному обсязі даних (717 fractured знімків). Для клінічного застосування необхідне збільшення датасету та можливе використання ансамблевих методів (TTA).

**3. Реалізовано виробниче рішення.** Наявність веб-інтерфейсу Gradio, модульна архітектура src/, конфігураційний файл та документація роблять проект готовим до подальшого розвитку.

**4. Підтверджено освітню цінність.** Проект охоплює повний цикл ML-розробки: аналіз даних, проектування архітектури, навчання з прогресивним збільшенням epochs, валідація та деплой у вигляді веб-застосунку.

Напрями вдосконалення системи:

| Напря́м                                  | Очікуваний ефект                   |
|------------------------------------------|------------------------------------|
| Розширення датасету (MURA, ChestX-Ray14) | Dice +0.10–0.15                    |
| Transformer backbone (Swin-T, ViT)       | Краще глобальне розуміння знімку   |
| Test-Time Augmentation (TTA)             | +0.02–0.05 Dice                    |
| Semi-supervised learning                 | Використання unlabeled X-ray даних |
| DICOM підтримка                          | Реальне медичне середовище         |

| Напря́м                | Очікуваний ефект             |
|------------------------|------------------------------|
| Uncertainty estimation | Довірчі інтервали для лікаря |

## ПЕРЕЛІК ПОСИЛАНЬ

1. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition. CVPR, 2016. DOI: 10.1109/CVPR.2016.90
2. Huang G., Liu Z., Van Der Maaten L., Weinberger K. Q. Densely Connected Convolutional Networks. CVPR, 2017. DOI: 10.1109/CVPR.2017.243
3. Liu Z., Mao H., Wu C.-Y. et al. A ConvNet for the 2020s. CVPR, 2022. arXiv:2201.03545
4. Caruana R. Multitask Learning. Machine Learning, 28(1), 41–75, 1997. DOI: 10.1023/A:1007379606734
5. Ronneberger O., Fischer P., Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI, 2015. arXiv:1505.04597
6. Yakubovskiy P. Segmentation Models Pytorch. [https://github.com/qubvel/segmentation\\_models.pytorch](https://github.com/qubvel/segmentation_models.pytorch)
7. Buslaev A. et al. Albumentations: Fast and Flexible Image Augmentations. Information, 11(2), 2020. DOI: 10.3390/info11020125
8. Abraham N., Khan N. A Novel Focal Tversky Loss Function With Improved Attention U-Net for Lesion Segmentation. ISBI, 2019. arXiv:1810.07842
9. Gradio Documentation. <https://gradio.app/>
10. FracAtlas: A Dataset for Fracture Classification, Localization and Segmentation. [https://figshare.com/articles/dataset/The\\_dataset/22363012](https://figshare.com/articles/dataset/The_dataset/22363012)

## ДОДАТОК А

### ПРАКТИЧНІ ДЕТАЛІ НАВЧАННЯ (З NOTEBOOK-ІВ)

- Для мультизадачного класифікатора використано клас `MultiTaskLoss` з вагами  $\lambda_{\text{bone}}=1.0$  і  $\lambda_{\text{frac}}=2.0$  (`BCEWithLogits` для обох задач).
- Метрики класифікації: `frac_recall`, `frac_f1`, `bone_f1` з `average='macro'` та `zero_division=0`.
- Сегментатор тренується лише на підмножині з переломами; батчі без переломів пропускаються, `Dice` та `IoU` рахуються з `smooth`-поправкою.
- Функція втрат сегментатора — `FocalTverskyLoss` ( $\alpha=0.7$ ,  $\beta=0.3$ ,  $\gamma=0.75$ ), що дає перевагу при сильному дисбалансі класів пікселів.
- Прогресивне навчання сегментатора: `v2` (`LR=5e-4`, 50 ер.)  $\rightarrow$  `v3` (+50 ер.)  $\rightarrow$  `v4` (+50 ер.)  $\rightarrow$  `v5` (`LR=2.5e-4`, +50 ер.).
- Checkpoint'и зберігаються у `notebooks/` як `fracture_segmenter_v*_best.pth` (найкраща валідація) та `fracture_segmenter_v*.pth` (фінал кожної стадії).
- Основний навчальний ноутбук: `notebooks/exploration_main.ipynb`.